

JavaScript Essentials



CS317 Mobile App Development

Lyn Turbak

Department of Computer Science
Wellesley College

Introduction

These slides summarize **some** key aspects of JavaScript that are especially useful for CS317 projects. I will continue to add to these slides as I think of more aspects.

Although they have a different surface syntax, JavaScript and Python are remarkably similar in many ways, but there are some important differences.

Some of the details in these slides may seem pedantic and nit-picky. But they're here for a reason: they will help you in your CS317 projects!

All examples shown here were run in node.js v18.17.1.

Primitive data types

```
> 17 + 25
```

```
42
```

```
> 3.14150 - 2.71
```

```
0.4315000000000002 // standard float issue in all languages
```

```
> "double quoted string" + ' single quoted string'
```

```
'double quoted string single quoted string'
```

```
> true || false // unlike Python, Boolean constant names are lowercase
```

```
true
```

```
> undefined // serves same role as Python's None value
```

```
undefined
```

```
> null // serves same role as Java's null value
```

```
null
```

Arithmetic operations

```
> 17 + 5 // Can be written w/o spaces: 17+5
22

> 17 - 5 // Can be written w/o spaces: 17-5
12

> 17*5
85

> 17/5
3.4

> Math.trunc(17/5) // integer division. Python's 17 // 3
3

> 17/0
Infinity // would be an error in Java, Python, C

> -42/0
-Infinity // would be an error in Java, Python, C

> 17%5 // remainder, just like Python
2

> 17%0
NaN // Not A Number floating point value. Would be an error in Java, Python, C

> 17**5
1419857 // exponentiation (175), just like Python
```

&& and || are short-circuit operations

If the first operand of && or || determines the result value, the result is returned **without** evaluating the second operand. In particular, if the second operand has an error, the && or || will **not** have an error in this case. This is known as short-circuit evaluation and is the same in Java, C, Python, etc.

```
// Assume variable blah is not defined
> blah
Uncaught ReferenceError: blah is not defined

> true || blah
true

> false || blah
Uncaught ReferenceError: blah is not defined

> false && blah
false

> true && blah
Uncaught ReferenceError: blah is not defined
```

Logical operations

```
> ! true // JS has Java's !, not Python's not
false

> ! false
true

> true && true // JS has Java's &&, not Python's and
true

> true && false
false

> true || false // JS has Java's ||, not Python's or
true

> false || false
false
```

Some string operations (more in String documentation)

```
> "computer " + 'science'
"computer science"

> 'ha'*3
NaN // Python-like multiplication doesn't work

> 'ha'.repeat(3)
"hahaha" // but can use .repeat

'science'.length // .length is a property
7

> 'science'[0]
"s" // 0-based indexing

> 'science'[4]
"n"

> 'science'[7]
undefined // not an error, but undefined value

> 'science'[-1]
undefined // No Python-like indexing from end

> 'science'.indexOf('c')
1 // index of first occurrence

> 'science'.indexOf('ce')
5

> 'science'.indexOf('a')
-1 // Couldn't find the substring
```

```
> 'sciences'.split('c')
['s', 'ien', 'es']

> 'li#17'.split('#')[1]
"17" // could be useful in Project 1

> 'generations'.slice(4)
'rations' // like Python's 'generations' [4:]

> 'generations'.slice(-4)
'ions' // like Python's 'generations' [-4:]

> 'generations'.slice(4,9)
'ratio' // like Python's 'generations' [4:9]

> 'generations'.slice(-4,-1)
'ion' // like Python's 'generations' [-4:-1]
```

JavaScript has complicated type coercion (implicit type conversion) rules!

```
> 32 + 10  
42
```

```
> 32 * 10  
320
```

```
> 32 * "10"  
320
```

```
> "32" *  
"10"  
320
```

```
> 32 + "10"  
'3210'
```

```
> 20 + true  
21
```

```
> "20" + true  
'20true'
```

```
> 20 * true  
20
```

```
> "20" * true  
20
```

```
> 20 + false  
20
```

```
> "20" + false  
'20false'
```

```
> 20 * false  
0
```

```
> "20" * false  
0
```

== (equal after type coercion) vs === (strict equality)

```
> 23 == 23
true

> 23 == 24
false

> 23 == 23.0
true

> 23 == 23.01
false

> 23 == "23"
true

> true == 1
true

> true == 1.0
true

> true == "1"
true

> true == "1.0"
true

'abc' == 'a' + 'bc'
true

> 'abc' == 'a ' + 'bc'
false
```

```
> 23 === 23
true

> 23 === 24
false

> 23 === 23.0
true

> 23 === 23.01
false

> 23 === "23"
false

> true === 1
false

> true === 1.0
false

> true === "1"
false

> true === "1.0"
false

'abc' === 'a' + 'bc'
true

> 'abc' === 'a ' + 'bc'
false
```

exp1 **!=** *exp2* means **!(exp1 == exp2)**

exp1 **!==** *exp2* means **!(exp1 === exp2)**

NaN is the “error number”

```
> 3 * "def"
```

```
NaN
```

```
> 12 * "abc"
```

```
NaN
```

```
> "d" * "ef"
```

```
NaN
```

```
> 3 + undefined
```

```
NaN
```

```
> 4 + NaN
```

```
NaN
```

```
> 5 * NaN
```

```
NaN
```

Variables and assignment: `let` and `const` declarations

Variables can be declared by `let` or `const`. The difference is that after a value is initially assigned to a `const`-bound variable, future assignments to that variable are prohibited.

```
> let a = 3 + 4
undefined

> a
7

> a = 10
10 // new value of a

> a = a*2
20 // new value of a
```

```
> const b = 3 + 4
undefined

> b
7

> b = 10
Uncaught TypeError: Assignment to constant variable.

> b = b* 2
Uncaught TypeError: Assignment to constant variable.
```

Note that `const` can name a mutable data structure (array, object) that is mutated because the value of the variable naming the data structure is the **same object**.

```
> const arr = [8, 3, 5];
undefined

> arr.push(2);
4

> arr[1] = 7;
7

> arr
[ 8, 7, 5, 2 ]
```

```
> const pt = {x:7, y:4};
undefined

> pt.x = 3
3

> pt.z = -5
-5

> pt
{ x: 3, y: 4, z: -5 }
```

Simple functions

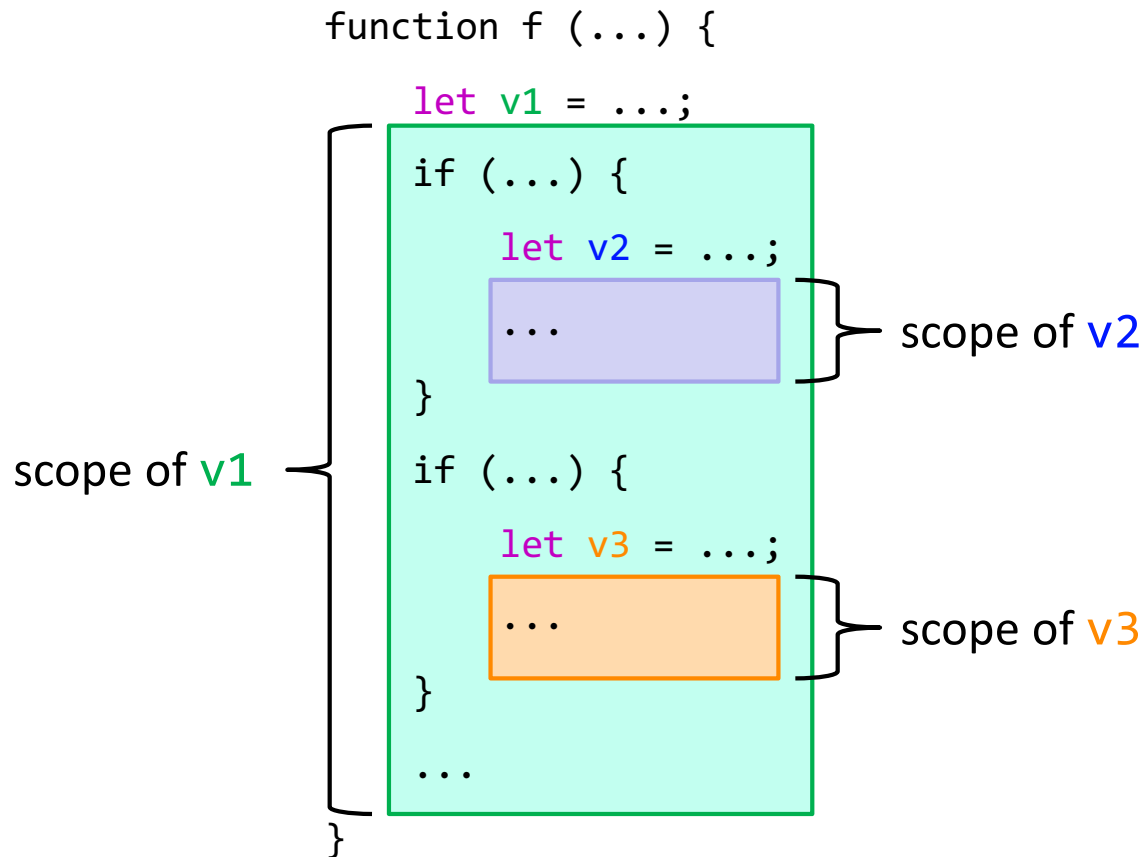
```
> function average(a,b) {  
... return (a+b)/2;  
... }  
undefined // function decl returns  
           // undefined value  
  
> average(10,15)  
12.5  
  
> average(10)  
NaN // NaN is floating point  
     // “Not a Number”. But why?  
  
> average(10,undefined)  
NaN // Ah, missing argument is *not* an  
     // error. It’s replaced by undefined!  
  
> average(10,15,20)  
12.5 // Extra arguments aren’t errors;  
     // they’re ignored!
```

```
> function noReturn(x,y) {  
... console.log(x + y);  
... }  
undefined  
  
> noReturn(3,4);  
7 // printed by console.log  
undefined // returned by noReturn!
```

Variable scope: **let** declarations

The **scope** of a variable is the area of a program in which it can be **referenced** (used).

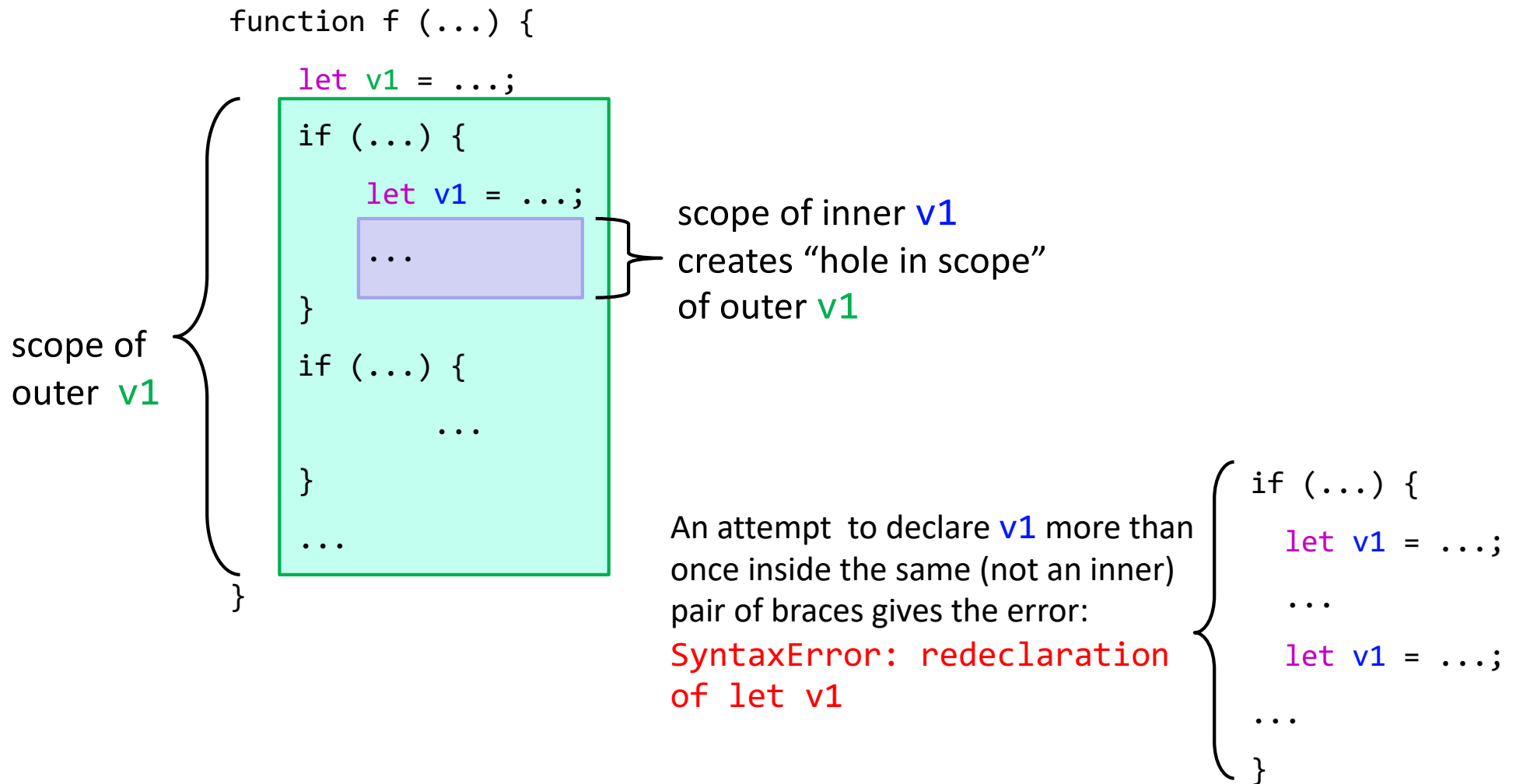
A variable declared (introduced) by **let** has **block scope**: it can be referenced right after it's declared until the end of the current block (usually terminated by **}**).



Attempting to use **v2** outside
its scope gives the error:
Uncaught ReferenceError:
v2 is not defined

Variable scope: redeclaring a variable with `let`

A variable declared by `let` can be redeclared in a smaller inner scope, creating a “hole in the scope” of the outer `let`-bound variable



Variable scope: **var** declarations

A variable declared by **var** has **function scope**: it **can** be referenced anywhere inside the current function, but will have the value **undefined** until it is assigned.

scope of **v1**,
v2, and **v3**

```
function f (...) {  
    var v1 = ...;  
    if (...) {  
        var v2 = ...;  
        ...  
    }  
    if (...) {  
        var v3 = ...;  
        ...  
    }  
    ...  
}
```

Implicit variable declarations (without a keyword) in Python behave like JavaScript's **var**-bound variables, not **let**-bound variables.

It's as if all **var** declarations are “hoisted” to the top of the function body, where they are declared to be undefined, and the former declarations become assignments.

```
function f (...) {  
    var v1 = undefined;  
    var v2 = undefined;  
    var v3 = undefined;  
    v1 = ...;  
    if (...) {  
        v2 = ...;  
        ...  
    }  
    if (...) {  
        v3 = ...;  
        ...  
    }  
    ...  
}
```

Because of this, a **var**-bound variable **can** be redeclared in the same scope, but it will mean the same function-scoped variable and will just be assigned a new value.

Variable scopes at top level of file

Variables can be declared at the top level of a file, outside of any function. These have the scope of the whole file, except for possible holes in scope within functions.

```
let v1 = ...  
function f (...) {  
    let v2 = ...;  
    if (...) { let v1 = ...; ... }  
    if (...) { let v3 = ...; ... }  
    ...  
}  
function g (...) {  
    let v1 = ...;  
    if (...) { let v2 = ...; ... }  
    if (...) { let v3 = ...; ... }  
    ...  
}
```

Variable scope: declarations without a keyword

Especially if you're a Python programmer, it's easy to forget to use a `let` or `var` keyword when introducing a variable. What happens then? They have **global scope** over the whole file (except if the names have been declared by `let`, `const`, or `var`).

This can lead to subtle bugs! So always declare your variables with a keyword (and use `let` rather than `var` unless you know what you're doing.)

scope of `v1`,
`v2`, and `v3`

```
function f (...) {  
    v1 = ...;  
    if (...) { v2 = ...; ... }  
    if (...) { v3 = ...; ... }  
    ...  
}  
  
function g (...) {  
    v1 = ...;  
    if (...) { v2 = ...; ... }  
    if (...) { v3 = ...; ... }  
    ...  
}
```

Truthy/Falsey values

```
> function truthyFalsey (val) {  
... return val ? 'truthy' : 'falsey'  
... }  
undefined  
  
> truthyFalsey(true)  
'truthy'  
  
> truthyFalsey(false)  
'falsey'  
  
> truthyFalsey(17)  
'truthy'  
  
> truthyFalsey(0)  
'falsey'  
  
> truthyFalsey(-1)  
'truthy'  
  
> truthyFalsey(NaN)  
'falsey'  
  
> truthyFalsey(Infinity)  
'truthy'
```

```
> truthyFalsey('hello')  
'truthy'  
  
> truthyFalsey('')  
'falsey'  
  
> truthyFalsey({x: 8, y: 5})  
'truthy'  
  
> truthyFalsey({})  
'truthy'  
  
> truthyFalsey([8, 5, 4])  
'truthy'  
  
> truthyFalsey([])  
'truthy'  
  
> truthyFalsey(null)  
'falsey'  
  
> truthyFalsey(undefined)  
'falsey'  
  
> truthyFalsey(Math.sqrt)  
'truthy'
```

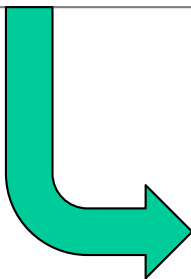
Simplifying Boolean Expressions and Conditionals

There are several code patterns involving boolean expressions and conditionals that can be simplified. The unsimplified versions are considered to be bad style and will be flagged in grading. Below **BE** stands for any expression evaluating to a boolean, and **STMS** stands for any statements.

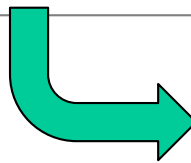
Complex Expr/Stmt	Simpler Expr/Stmt	Complex Expr/Stmt	Simpler Expr/Stmt
BE === True	BE	BE === False	!BE
if (BE) return true ; else return false ;	return BE ;	if (BE) return false ; else return true ;	return !BE ;
if (BE1) return BE2 ; else return false ;	return BE1 && BE2 ;	if (BE1) return true ; else return BE2 ;	return BE1 BE2 ;
if (BE) STMS return true ; else STMS return false ;	STMS return BE ;	result = BE ; return result ;	return BE ;

Simplifying Boolean Expressions and Conditionals: Example

```
function doesNotBeginWithVowel(s) {  
  if (isVowel(s[0]) == False)  
    return true;  
  else  
    return false;  
}
```



```
function doesNotBeginWithVowel(s) {  
  if (!isVowel(s[0]))  
    return true;  
  else:  
    return false;  
}
```



```
function doesNotBeginWithVowel(s) {  
  return !isVowel(s[0])  
}
```

Recursion works as expected. But integers do not!

```
> function factorial(n) {  
...   if (n == 0) {  
...     return 1;  
...   } else {  
...     return n * factorial(n-1);  
...   }  
... }
```

Undefined

```
> factorial(5)
```

120

```
> factorial(21)
```

51090942171709440000

// this is an integer

```
> factorial(22)
```

1.1240007277776077e+21

// this is a floating point value. Why?

```
> factorial(170)
```

7.257415615307994e+306

```
> factorial(170)
```

7.257415615307994e+306

```
> factorial(171)
```

Infinity // floating point +infinity

```
> factorial(170) / factorial(169)
```

170

```
> factorial(171) / factorial(170)
```

Infinity // numerator is infinity,
// but denominator is finite

```
> factorial(172) / factorial(171)
```

NaN // both numerator and denominator
// are infinity, so division
// is undefined

BigInt supports arbitrarily large integers

You're unlikely to need this in CS317, but you might as well know.

Also, this differs from Python, where big integers are the default.

```
> BigInt(17)
17n // n at the end means it's a BigInt
> 17n // n notation for BigInt literals
17n
> function bigFact(n) {
...   if (n == 0) {
...     return 1n;
...   } else {
...     return BigInt(n) * bigFact (n-1);
...   }
... }
undefined
> bigFact(5)
120n
> bigFact(22)
1124000727777607680000n
```

```
> bigFact(171)
124101807021766782342484052410310399261\
660557750169318538895180361199607522169\
175299275197812048758557646495950167038\
705280988985869071076733124203221848436\
431047357788996854827829075454156196485\
215346831804429323959817369689965723590\
394761615227855818006117636510842880000\
000000000000000000000000000000000000n
> bigFact(171) / factorial(170)
171
> bigFact(172) / bigFact(171)
172
```

Essential debugging function: console.log

JavaScript's `console.log` is like Python's `print` and Java's `System.out.println`

```
function testConsoleLog(n) {  
  if (n > 0) {  
    console.log(`${n} is positive`);  
  } else if (n < 0) {  
    console.log(`${n} is negative`);  
  } else {  
    console.log(`${n} is zero`);  
  }  
  if (n % 2 == 0) {  
    console.log(`${n} is even`);  
  } else {  
    console.log(`${n} is odd`);  
  }  
  console.log(n, n/2, 7*n, (n**2).toString());  
  let arr = [n, n/2, 7*n, (n**2).toString()];  
  console.log(arr);  
  console.log([n, n/2], [7*n, (n**2).toString()]);  
  console.log(`Array is ${arr}`)  
  console.log(`Array is ${JSON.stringify(arr)}`)  
}
```

```
> testConsoleLog(9)  
9 is positive  
9 is odd  
9 4.5 63 81  
► Array(4) [ 9, 4.5, 63, "81" ]  
// in browser debuggers, can click ►  
// to expand details.  
► Array [ 9, 4.5 ] ► Array [ 63, "81" ]  
Array is 9,4.5,63,81  
// notational details lost here  
Array is [9,4.5,63,"81"]  
// JSON.stringify shows usual notations
```

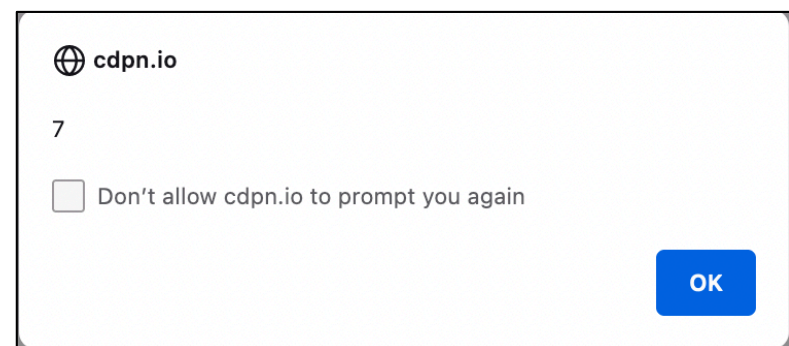
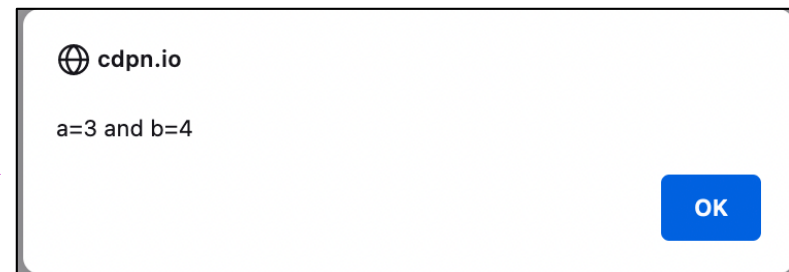
```
testConsoleLog(-4)  
-4 is negative  
-4 is even  
-4 -2 -28 16  
► Array(4) [ -4, -2, -28, "16" ]  
► Array [ -4, -2 ] ► Array [ -28, "16" ]  
Array is -4,-2,-28,16  
Array is [-4,-2,-28,"16"]
```

Helpful debugging function: alert

`alert(value)` pops up a dialog window displaying *value* suspending the computation until the user clicks on the window. This can be helpful for slowing things down and seeing the order of execution.

Alert only works in browser-based JavaScript implementations, not node.js

```
function testAlert(a,b) {  
  alert(`a=${a} and b=${b}`);  
  alert(a + b);  
}  
  
testAlert(3, 4);
```



JavaScript Objects

To a first approximation, JavaScript objects are like Python dictionaries: collections of mutable key: value pairs. (They're much more complicated, but more on that later.)

```
> let pt = {x:7, y:4}
undefined

> pt.x
7

> pt['y'] // alternative syntax; all keys
4         // are really strings

pt.z
undefined // not an error

> pt.x = 3 // can change value for key
3 // assignment returns new value

> pt
{ x: 3, y: 4 }

> pt.z = -5 // can add new key/value pair
-5

> pt
{ x: 3, y: 4, z: -5 }
```

```
> let stud = {name: 'Georgia', age: 17, registered: true}
// values can have any type
undefined

> let obj1 = {'a b': 5, 42: 3.141, true: 'dog'}
// keys can be other values, but converted to strings
undefined

> Object.keys(obj1)
[ '42', 'a b', 'true' ]

> obj1['a b']
5

> obj1[42]
3.141

> obj1['42']
3.141

> obj1[true]
'dog'

> obj1['true']
'dog'
```

JavaScript Arrays

JavaScript arrays are mutable, extensible, indexed data structures that are similar to Python lists, with a few twists.

```
> let a1 = [8, 3, 5]
undefined

> let a2 = [3.13, true, 'cat', a1]
/* As in Python, JS arrays are
   homogeneous, i.e., can contain
   elements of different types */
undefined

> a1.length
3

> a2.length
4

> a1[0]
8

> a2[2]
'cat'

> a2[3][2] // nested indexing
5
```

```
> a2[4]
undefined // *not* an index error!

> a1[-1]
undefined // *not* an index error!

> a1[1] = 9
9 // assignment returns new value

> a1
[ 8, 9, 5 ] // array slots are mutable

> a2 // illustrates sharing of a1
[ 3.13, true, 'cat', [ 8, 9, 5 ] ]

> a2[3][2] = 'ant'
'ant'

> a2 // illustrates sharing of a1
[ 3.13, true, 'cat', [ 8, 9, 'ant' ] ]
```

Assigning to out-of-range array slots

Elements can be assigned to out-of-range slots, leading to **empty slots** in between.

```
> a1
[ 8, 9, 'ant' ]

> a1[3] = 17 /* can extend array by
assigning to out-of-range index */
17

> a1
[ 8, 9, 'ant', 17 ]

> a1.length
4

> a1[5] = 2
2

> a1
[ 8, 9, 'ant', 17, <1 empty item>, 2 ]

> a1.length
6

> a1[8] = 42
42

> a1
[ 8, 9, 'ant', 17, <1 empty item>, 2,
  <2 empty items>, 42 ]

> a1.length
9
```

```
> Object.keys(a1) // arrays are just special objects
[ '0', '1', '2', '3', '5', '8' ]

> Array.from(a1.keys())
[ 0, 1, 2, 3, 4, 5, 6, 7, 8 ]

> a1[-1] = 57
57

> a1['cat'] = 'dog'
'dog'

> a1
[ 8, 9, 'ant', 17, <1 empty item>, 2,
  <2 empty items>, 42, '-1': 57, cat: 'dog' ]

> a1.length
9

> Object.keys(a1)
[ '0', '1', '2', '3', '5', '8', '-1', 'cat' ]

> Array.from(a1.keys())
[ 0, 1, 2, 3, 4, 5, 6, 7, 8 ]

> let a2 = [8, 9, 'ant', 17, , 2, , , 42]
undefined // spaces between commas denote empty slots

> a2
[ 8, 9, 'ant', 17, <1 empty item>, 2, <2 empty items>, 42 ]
```

Some other array operations (see Array docs for more)

```
> a1 = [8, 3, 5]
[ 8, 3, 5 ]

> a1.push(2) // like Python .append
4 // returns length of updated list

> a1
[ 8, 3, 5, 2 ]

> a1.push(9,7) // can push multiple vals
6

> a1
[ 8, 3, 5, 2, 9, 7 ]

> a1.pop() // like Python .pop, but
7          // can't specify index

> a1
[ 8, 3, 5, 2, 9 ]
```

```
> a1.shift()
8

> a1
[ 3, 5, 2, 9 ]

> a1.unshift(7,1)
6

> a1
[ 7, 1, 3, 5, 2, 9 ]

> a1 + [8,6]
'7,1,3,5,2,98,6' // + doesn't concat lists in JS!

> a1.concat(8,6)
[ 7, 1, 3, 5, 2, 9, 8, 6 ]

> a1
[ 7, 1, 3, 5, 2, 9 ]
```

Spread notation (...) for arrays

Suppose *arrExp* is an expression that denotes an array with *n* elements.
Then *...arrExp* can be used in any context that expects *f* comma-separated values.

```
> let a1 = [8, 3, 5]
undefined

> let a2 = [6, 4]
undefined

> [7, 1, ...a1]
[ 7, 1, 8, 3, 5 ]

> [...a2, 9, 2]
[ 6, 4, 9, 2 ]

> [...a1, ...a2]
[ 8, 3, 5, 6, 4 ]

> [7, ...a2, 1, ...a1, 9, ...a2, 2]
[ 7, 6, 4, 1, 8, 3, 5, 9, 6, 4, 2 ]
```

```
> a3.push(a2)
4

> a3
[ 7, 1, 9, [ 6, 4 ] ]

> a3.push(...a2)
6

> a3
[ 7, 1, 9, [ 6, 4 ], 6, 4 ]

> console.log(a1)
[ 8, 3, 5 ]
undefined

> console.log(...a1)
8 3 5
undefined
```

typeof and Array.isArray

```
> typeof(17)
'number'

> typeof(3.141)
'number' // ints and floats
          // are not distinguished

> typeof(Infinity)
'number'

> typeof(NaN)
'number'

> typeof('cat')
'string'

> typeof(true)
'boolean'

> typeof(Math.sqrt)
'function'

> typeof(undefined)
'undefined'
```

```
> typeof({x: 3, y: 7})
'object'

> typeof([8, 3, 5])
'object' // arrays are not distinguished
          // from other objects

> Array.isArray([8, 3, 5])
true // use Array.isArray to
     // test for arrays

> Array.isArray({x: 3, y: 7})
false

> typeof(null)
'object'
```

Anonymous functions and arrow notation

Each group below begins with a **named function**.

The assignments illustrate notations for equivalent **anonymous functions**, including **arrow notation** (`=>`), which can replace `return expression` by *expression* if there's only one expression to the right of the arrow.

```
function sq1 (n) { return n*n; }  
let sq2 = function (n) { return n*n; }  
let sq3 = n => { return n*n; }  
let sq4 = n => n*n;
```

```
function avg1 (a,b) { return (a+b)/2; }  
let avg2 = function (a,b) { return (a+b)/2; }  
let avg3 = (a,b) => { return (a+b)/2; }  
let avg4 = (a,b) => (a+b)/2;
```

```
function logSum1 (a,b) { console.log(a,b); return a+b; }  
logSum2 = function (a,b) { console.log(a,b); return a+b; }  
logSum3 = (a,b) => { console.log(a,b); return a+b; }
```

Anonymous functions can replace a function name position when calling a function. E.g.:

```
(n => n*n)(9) evaluates to 81
```

Special case of returning objects from arrow functions

```
> const makePointBroken = (xcor, ycor) => {x: xcor, y: ycor}  
const makePointBroken = (xcor, ycor) => {x: xcor, y: ycor}
```

^

Uncaught SyntaxError: Unexpected token ':'

The problem with the above is that curly braces after `=>` are interpreted as function body, and the function `function (a, b) {x: xcor, y: ycor}` is syntactically invalid.

Below are two techniques to fix this problem when returning an object from an arrow function.

// Technique #1: use an explicit return

```
> const makePointFixed1 = (xcor, ycor) => {return {x: xcor, y: ycor}}  
undefined  
  
> makePointFixed1(3,4)  
{ x: 3, y: 4 }
```

// Technique #2: wrap object curly braces in parens

```
> const makePointFixed2 = (xcor, ycor) => ({x: xcor, y: ycor})  
undefined  
  
> makePointFixed2(5,6)  
{ x: 5, y: 6 }
```

Other weirdnesses involving functions that return objects

Object properties are normally expressed in curly braces via the syntax *propName: propExpr*

But just using *propName* is a shorthand for *propName: propName*

```
> function objReturn1(v1, v2, v3) {return {prop1: v1, v2, prop3: v3}}
undefined

> objReturn1('cat', 'dog', 'emu')
{ prop1: 'cat', v2: 'dog', prop3: 'emu' }
```

Sometimes you want to use an expression in place of the property name.

To do so, wrap that expression in square brackets.

```
> function objReturn2(v1, v2, v3) {return {prop1: v1, [v2]: v3}}
undefined

> objReturn2('cat', 'dog', 'emu')
{ prop1: 'cat', dog: 'emu' }

> function objReturn3(v1, v2, v3) {return {[v1+v2]: v3}}
undefined

> objReturn3('cat', 'dog', 'emu')
{ catdog: 'emu' }
```

JavaScript functions are first class

In a programming language, a value is said to be **first class** if it has the following properties:

1. It can be assigned to a variable.
2. It can be passed to a function as an argument.
3. It can returned from a function as a result.
4. It can be stored as a value in a data structure
(e.g., in JavaScript: as the value of an array slot, as the value for a key in an object)
5. It can be created in any context.

Functions in JavaScript are first class!

- Previous slide shows that functions can be assigned to variables.
- Slides following this one will illustrate other properties.

Notes:

- Before the 1990s, relatively few languages (mostly dialects function-oriented languages like Lisp, ML, and Haskell) had first-class functions. But now many do, including JavaScript, Python, (to some extent) Java, and even Excel spreadsheets (with the new LAMBDA feature).
- C has the first 4 properties, but does not have the 5th , because functions cannot be created inside other functions.
- Lyn considers first-classness of functions the most important feature a programming language can have, because you can perform magic with it!

Functions can be passed as arguments to other functions

```
function applyTo5 (f) {  
  return f(5);  
}
```

```
> applyTo5(sq1)  
??A1  
  
> applyTo5(n => n*10)  
??A2  
  
> applyTo5(x => x**3)  
??A3
```

```
function applyTo3And7 (f) {  
  return f(3,7);  
}
```

```
> applyTo3And7(avg1)  
??A4  
  
applyTo3And7(logSum1)  
??A5  
  
> applyTo3And7( (m,b) => m*10 + b )  
??A6
```

Functions can be returned as results from other functions

```
function makeSub(n) {  
  return (x => x - n);  
}
```

```
function makeLinear(m, b) {  
  return (x => m*x + b);  
}
```

```
> makeSub(7)(10)  
??B1  
  
> applyTo5(makeSub(3))  
??B2  
  
> applyTo5(makeSub)(3)  
??B3  
  
> applyTo5(applyTo5(makeSub))  
??B4
```

```
> makeLinear(10,2)(4)  
??B5  
  
> applyTo5(makeLinear(10,2))  
??B6  
  
> applyTo3And7(makeLinear)(10)  
??B7  
  
> applyTo5(applyTo3And7(makeLinear))  
??B8
```

Important: Note that the values of the parameters `n` in `makeSub` and `m, b` in `makeLinear` are in scope in the bodies of the returned functions and are "remembered" even though the function frame that created the returned function is no longer active! So `makeSub(7)` acts like `(x => x - 7)` and `makeLinear(10,2)` acts like `(x => 10*x + 4)`.

Functions can be stored in data structures (arrays, objects)

```
let funcsArr = [  
  sq1,  
  n => n**3,  
  applyTo5(makeSub),  
  makeLinear(10,2),  
  applyTo3And7(makeLinear)  
]
```

```
> funcsArr[0](9)  
??C1  
  
> funcsArr[1](9)  
??C2  
  
> funcsArr[2](9)  
??C3  
  
> funcsArr[3](9)  
??C4  
  
> funcsArr[4](9)  
??C5
```

```
let funcsObj = {  
  sq: sq1,  
  cube: n => n**3,  
  sub_5: applyTo5(makeSub),  
  linear_10_2: makeLinear(10,2),  
  linear_3_7: applyTo3And7(makeLinear)  
}
```

```
> funcsObj.sq(9)  
??C6  
  
> funcsObj.cube(9)  
??C7  
  
> funcsObj.sub_5(9)  
??C8  
  
> funcsObj.linear_10_2(9)  
??C9  
  
> funcsObj.linear_3_7(9)  
??C10
```

Functions can be created in any context

An expression that creates a function can be put in anywhere in JavaScript where an expression is expected. This has already been illustrated in the previous slides, but below are a few more examples.

```
function chooseFun(bool) {  
  return bool ? (n => n - 3) : (x => x**3);  
}
```

```
> chooseFun(3 === 3)(10)  
??D1
```

```
> chooseFun(4 == 5)(10)  
??D2
```

```
const makeLinearStaged = m => (b => (x => m*x + b)) // a so-called "Curried" function  
const applyTo3And7Staged = f => f(3)(7)
```

```
> applyTo5(applyTo3And7Staged(makeLinearStaged))  
??D3
```

```
> applyTo3And7Staged(applyTo5(makeLinearStaged))  
??D4
```

JavaScript: mapping and filtering lists

.map and .filter create **new** lists, which is important for React's requirement that changes to state should not mutate existing structure but create new structure.

```
> const pts = [{x:3, y:8}, {x:1, y:6}, {x:10, y:7}, {x:5, y:9}, {x:4, y:2}]
undefined

> pts.map( pt => pt.x )
[3, 1, 10, 5, 4]

> pts.map( pt => pt.x + pt.y )
[ 11, 7, 17, 14, 6 ]

> pts.filter( pt => pt.x%2 == 1 )
[ { x: 3, y: 8 }, { x: 1, y: 6 }, { x: 5, y: 9 } ]

> pts.filter( pt => pt.x + pt.y > 10 )
[ {x: 3, y: 8}, {x: 10, y: 7}, {x: 5, y: 9} ]

> pts.filter( pt => pt.x + pt.y > 10 ).map( pt => pt.x )
[ 3, 10, 5 ]

// Write an expression that gives a list of the points in pts
// with swapped x/y coords when the product of their coords is even
> ??expr1
[ { x: 8, y: 3 }, { x: 6, y: 1 }, { x: 7, y: 10 }, { x: 2, y: 4 } ]
```

.find returns first element satisfying predicate

```
> pts
[{x:3, y:8}, {x:1, y:6}, {x:10, y:7}, {x:5, y:9}, {x:4, y:2}]

> pts.find( pt => pt.x < 2 )
{x:1, y:6}

> pts.find( pt => pt.x >= 5 )
{x:10, y:7}

> pts.find( pt => pt.x + pt.y < 7 )
{x:4, y:2}

> pts.find( pt => pt.x < 0 )
undefined // undefined returned if no item satisfies predicate

> pts.find( pt => pt.x + pt.y < 7 ).x
4

> pts.filter( pt => pt.x + pt.y < 7 ).x
undefined // singleton list [{x:4, y:2}] has no x field

> pts.filter( pt => pt.x + pt.y < 7 )[0].x
4 // .find is clearer!

// Write an expression finds the first point in pts with odd y coord
// whose x coord is smaller
> ??expr2
{ x: 5, y: 9 }
```

.every and .some

elts.every(*predicate*) returns true if *predicate* is true for every elt in *elts*; otherwise false.

```
> pts
[{x:3, y:8}, {x:1, y:6}, {x:10, y:7}, {x:5, y:9}, {x:4, y:2}]
> pts.every( p => p.y > 1 )
true
> pts.every( p => p.x > 1 )
false
> pts.every( p => p.x > p.y )
false
```

elts.some(*predicate*) returns true if *predicate* is true for at least one elt in *elts*; otherwise false.

```
> pts.some( p => p.x > 9 )
true
> pts.some( p => p.y > 9 )
false
> pts.some( p => p.x > p.y )
true
```

.map, .filter, .find, .every, .some functions can take 2 extra args

After the 1st arg (the array element), the 2nd arg is the index, and the 3rd arg is the array itself. The following examples show why this is useful.

```
> pts
[{x:3, y:8}, {x:1, y:6}, {x:10, y:7}, {x:5, y:9}, {x:4, y:2}]

> pts.filter( (pt, i) => (i >=2) && pt.y > 5)
[ {x:10, y:7}, {x:5, y:9} ]

> pts.find( (pt, i) => (i >=2) && pt.y < 7).x
4

> pts.find( (pt, i, arr) => (i >=2) && pt.y > 5 && arr[i-1].y > 6 )
{x:5, y:9}

> pts.map( pt => pt.x)
[3, 1, 4, 10, 5]

> pts.map( pt => pt.x ).map( (pt, i, a) => (i===0) ? a[a.length-1] : a[i-1]) )
[5, 3, 1, 4, 10]
```

reducing arrays

`[v1, v2, ..., vn].reduce( fcn, init )` returns `fcn(..., fcn(fcn(init, v1), v2), ... , vn)`

```
> const strs = 'I went to the store'.split(' ')
> strs
[ 'I', 'went', 'to', 'the', 'store' ]
> strs.reduce( (strSoFar, st) => st+strSoFar, '' )
'Iwenttothestore'
> strs.reduce( (strSoFar, st) => strSoFar + '! ' + st, '$$' )
'store! the! to! went! I! $$'
> strs.map(st => st.length).reduce( (sum, num) => num+sum, 0 )
15
// What are ??val1 and ??val2?
> strs.map(s => s.length).reduce( (m, n) => Math.max(m,n), 0 )
??val1
> strs.reduce( (t, s1) => s1.length > t[1] ? [s1, s1.length] : t, ['', 0] )[0]
??val2
// Write an expression that denotes the product of the y coords in pts
// that are less than the x coords
// const pts = [{x:3, y:8}, {x:1, y:6}, {x:10, y:7}, {x:5, y:9}, {x:4, y:2}]
> ??expr2
14
```

.forEach

elts.forEach(*fcn*) applies *fcn* to every element in *elts*

```
> pts
[{x:3, y:8}, {x:1, y:6}, {x:10, y:7}, {x:5, y:9}, {x:4, y:2}]

> pts.forEach ( p => {if (p.x > 4) {console.log(p.x);}} )
10 // first printed p.x
5  // second printed p.x
undefined // result *returned* by pts.forEach(...)

// An alternative version of above expression
> pts.map( p => p.x ).filter( x => x > 4 ).forEach(x => console.log(x))
10 // first printed p.x
5  // second printed p.x
undefined // result *returned* by the expression
```

You are **not** allowed to use **for** loops in CS317!

- because you can instead use `.map`, `.filter`, `.find`, `.every`, `.some`, `.sort`, etc. instead.
- you should use `.forEach` sparingly, when you cannot just use the above methods.
- you **may** use while loops when the above methods are insufficient

```
function sumSquaredOdds(ints) {  
  sum = 0;  
  for (i = 0; i < ints.length; i++) {  
    if (ints[i] % 2 == 1) {  
      sum += ints[i] ** 2;  
    }  
  }  
  return sum;  
}
```

```
// Express the above function in arrow notation  
const sumSquaredOdds = ints =>
```

JavaScript: Array identity, sharing, mutation. and copying

Important because React JS uses identity to determine when state values have *changed*

```
> L1 = [3, 1, 7]
[ 3, 1, 7 ]
> L2 = L1
[ 3, 1, 7 ]
> L1 === L2 // same as L1 == L2 for lists
true // the same list in memory!
> L3 = L1.slice(0,3) // copy the list
[ 3, 1, 7 ]
> L1 === L3
false // different lists in memory!
> L4 = [... L1]
// ... "spread" notation copies list
[ 3, 1, 7 ]
> L1 == L4
false // different lists in memory!
> L5 = [].concat(L1) // copy the list
[ 3, 1, 7 ]
> L1 === L5
false // Are different lists in memory
```

```
> LL1 = [L1, L2, L3, L4, L5]
[ [ 3, 1, 7 ], [ 3, 1, 7 ],
  [ 3, 1, 7 ], [ 3, 1, 7 ],
  [ 3, 1, 7 ] ]
> L1.push(8) // append 8 to the
              // end of L1
4 // returns the new length
> LL1
[ [ 3, 1, 7, 8 ], [ 3, 1, 7, 8 ],
  [ 3, 1, 7 ], [ 3, 1, 7 ],
  [ 3, 1, 7 ] ]
// push to L1 changes L2 but
// not L3, L4, or L5
> LL1[0] === LL1[1]
true
> LL1[0] === LL1[2]
false
> LL1[0] === LL1[3]
false
> LL1[0] === LL1[4]
false
```

JavaScript: Array copying is shallow

Shallow copies of a container copy only the container and **not** the contained elements.

```
> LL2 = [... LL1]
[ [ 3, 1, 7, 8 ], [ 3, 1, 7, 8 ], [ 3, 1, 7 ],
  [ 3, 1, 7 ], [3, 1, 7] ]
> LL2[0] == LL2[1]
true // copy was a *shallow* (1-level), not *deep*
> LL2[0] == LL2[2]
false
> LL2[0] == LL2[3]
false
> LL2[0] == LL2[4]
false
```

JavaScript: copying objects, with changes

...“spread” notation can be used in objects (as well as arrays) for shallow copying

```
> pt1 = {x:1, y:2, z:3}
{x:1, y:2, z:3}

> pt2 = {... pt1}
  // ... “spread” notation copies object;
  // pt2 is a (shallow) copy of pt1
{x:1, y:2, z:3}

> pt1 === pt2
false

> {... pt1, x:4} // returns a copy of pt1 with x changed
{x:4, y:2, z:3}

> {... pt1, y:5} // returns a copy of pt1 with y changed
{x:1, y:5, z:3}

> {... pt1, x:6, z:7} // returns a copy of pt1 with x and z changed
{x:6, y:2, z:7}
```

JavaScript: mapping to updated objects

```
> pts
[ { x: 10, y: 7 }, { x: 3, y: 8 }, { x: 1, y: 6 },
  { x: 4, y: 2 }, { x: 7, y: 9 } ]

// The following is tricky because:
// (1) Outermost { ... } after => is JavaScript “sequence of
//     statements”, *not* “make an object”
// (2) Because of (1) need an explicit return keyword!

> pts.map (pt => { return {... pt, x:pt.x*2} } )
[ { x: 20, y: 7 }, { x: 6, y: 8 }, { x: 2, y: 6 },
  { x: 8, y: 2 }, { x: 14, y: 9 } ]
```

JavaScript: JSON.stringify & JSON.parse

```
> obj1 = {int: 17, float: 3.14, bool: true, array: [23, 2.71, false], pt: {x: 7, y: 4}}
obj1 = {int: 17, float: 3.14, bool: true, array: [23, 2.71, false], pt: {x: 7, y: 4}}
{
  int: 17,
  float: 3.14,
  bool: true,
  array: [ 23, 2.71, false ],
  pt: { x: 7, y: 4 }
}
> string1 = JSON.stringify(obj1)
string1 = JSON.stringify(obj1)
'{"int":17,"float":3.14,"bool":true,"array":
  [23,2.71,false],"pt":{"x":7,"y":4}}'
> string2 = JSON.stringify(obj1, null, 2)
string2 = JSON.stringify(obj1, null, 2)
'{\n' +
  '  "int": 17,\n' +
  '  "float": 3.14,\n' +
  '  "bool": true,\n' +
  '  "array": [\n' +
  '    23,\n' +
  '    2.71,\n' +
  '    false\n' +
  '  ],\n' +
  '  "pt": {\n' +
  '    "x": 7,\n' +
  '    "y": 4\n' +
  '  }\n' +
  '}'
```

```
> obj2 = JSON.parse(string1)
obj2 = JSON.parse(string1)
{
  int: 17,
  float: 3.14,
  bool: true,
  array: [ 23, 2.71, false ],
  pt: { x: 7, y: 4 }
}
> obj3 = JSON.parse(string2)
obj3 = JSON.parse(string2)
{
  int: 17,
  float: 3.14,
  bool: true,
  array: [ 23, 2.71, false ],
  pt: { x: 7, y: 4 }
}
```

Variable declaration bug example

The following example shows how replacing **let** by **var** or omitting **let** can lead to bugs.
(Try this example at <https://cs.wellesley.edu/~cs317/code/javascript/scope/scopeBugs.html>).

```
/**
 * Return a nullary function that prints n
 */
function makePrintFn(n) {
  return () => console.log(n);
}

/**
 * Function illustrating importance of let
 * vs. var (or omitting a declaration keyword).
 */
function scopeExample() {
  printFns = [];
  for (i = 0; i <= 3; i++) {
    // Changing let to var or omitting it causes bug!
    let printFn = makePrintFn(i);
    printFns.push ( () => printFn() );
  }
  console.log("scopeExample:");
  printFns.forEach( printer => printer() );
}
```

As written, `scopeExample()` displays:

```
0
1
2
3
```

But if the **let** is changed to **var** or is omitted, `scopeExample()` displays:

```
3
3
3
3
```